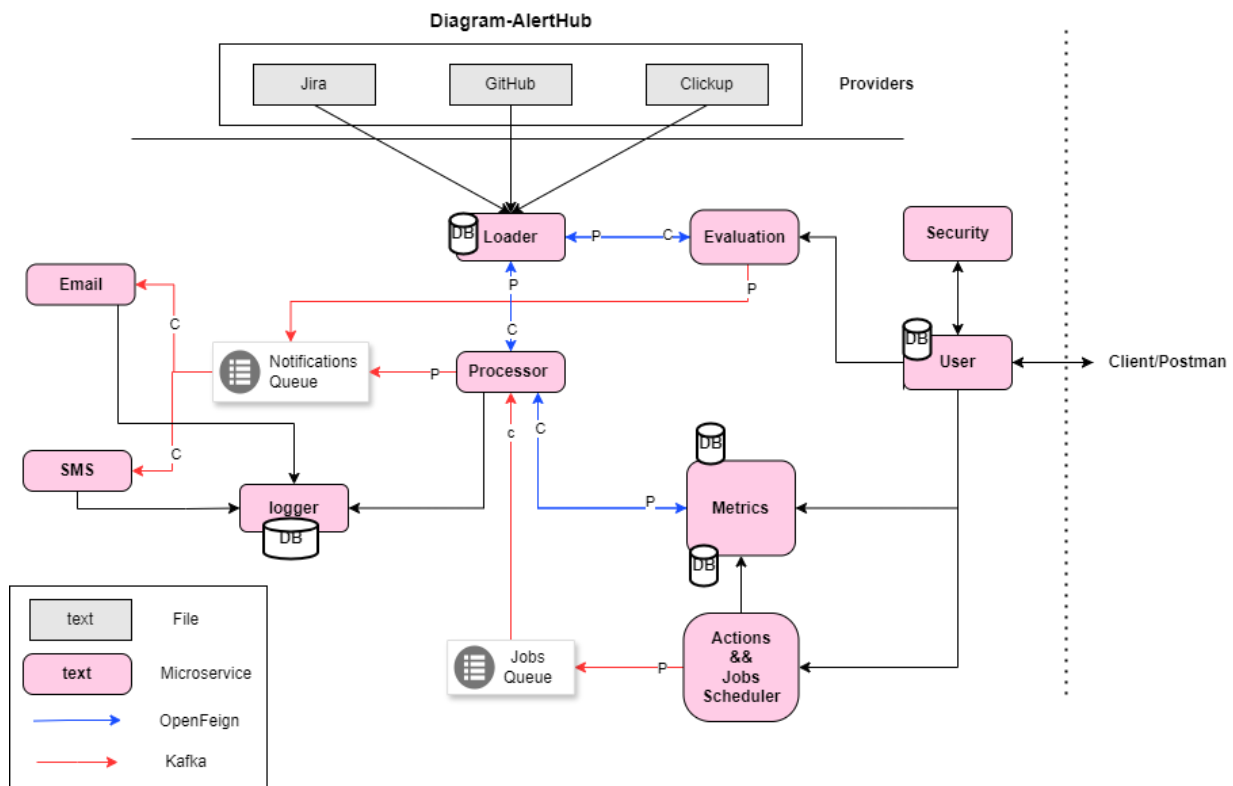


Alert Hub

The system notifies the project manager about changes recorded in the Task Management Tool (such as Jira, ClickUp, or GitHub). Notifications can be sent via SMS or Email.

Diagram:



The application consists of 10 microservices:

1. Loader
2. Security
3. Action
4. Metric
5. Processor
6. Sender Email
7. Sender SMS
8. User
9. Evaluation
10. Logger

Requirements for each microservice (logger not required):

- 1- Testing service layer

- 2- Handle exceptions
 - 3- Implement Swagger (OpenAPI) documentation
-

Providers:

Structure:

There are three types of data from different platforms

1. GitHub
2. Jira
3. ClickUp

Each type has a folder in the ([data](#)) directory with the same name. Inside each folder, there is a data file.

file name format:{provider_name}_yyyy_mm_ddThh_mm_ss

Example: jira_2024_08_22T13_30_00

Loader Services:

The functionality of loader to read the correct file (Need to be synch with the latest scanned files)

extract the data, transform it to match the DB schema, and store it in the DB into platformInformation table Each loader will automatically run on an **hourly** basis to check for new data.

Also, add the ability to trigger scanning files manually.

Mapping Schema :

GitHub	ClickUp	Jira	DB Column name
[Timestamp of now]	[Timestamp of now]	[Timestamp of now]	timestamp
manager_id	owner_id	manager_id	owner_id
projects	project	projects	project
assignee	tag	assignee	tag
label	label	label	label
devloper_id	worker_id	employeeID	developer_id

issue	task	issue	task_number
environment	pr_env	env	environment
user_story	user_story	user_story	user_story
point	day	point	task_point
sprint	currant_sprint	sprint	sprint

Notes:

- 1- Pay attention to empty or Null values, in case of numeric value, Null will be treated as 0
- 2- Timestamp column in the DB will be the time of the scan

DataBase (MySQL)

Tables:

- 1- users (there are 3 tables to implement)
 - 2- platformInformation
 - 3- metric
 - 4- action
-

Security Services:

The service will have access control and will manage users and their permissions.

Action in the application that needs permission:

- **Create action**
- **Update action**
- **Delete action**
- **Create Metric**
- **Update Metric**
- **Delete Metric**
- **Trigger manual process (action)**

Description : This trigger allows a user to bypass all predefined conditions and immediately run the check for a specific action.

- **Trigger manual scan**

Description: This trigger enables the user to manually initiate a scan for new data files, instead of waiting for the automated process to run at its scheduled time.

- **Trigger evaluation**

Description: This trigger provides the manager with detailed insights into a developer's performance and workload. It offers information about the developer's tasks and performance metrics, helping managers make informed decisions.

- **Read** (Get All action and metric)

All users have read permission, user can have permission for multiple or all of the list above.
* Admin Permission: Admin have all permission of the list above additionally can create new users, delete existing users, and grant or revoke permissions.

user:

id	username	email	phone	password
1	admin	admin@gmail.com	0509999999	admin1234
1001	Ameen	Ameen@gmail.com	0523456789	secret

role:

id	role
1	"createAction"
2	"updateAction"
3	"deleteAction"
4	"createMetric"
5	"updateMetric"
6	"deleteMetric"
7	"triggerScan"
8	"triggerProcess"
9	"triggerEvaluation"
10	"read"

user_role

id	user_id	role_id
1	1001	1
2	1001	2
3	1001	3
4	1001	7

Metric Services:

id	user_id	name	label	threshold	time_frame_hours
1	1001	help_1_4	helpWanted	1	4
2	1001	bug_10_12	bug	10	12
3	1001	first_1_24	good_first_issue	1	24

Example: (line 2) If there are 10 tasks in a project labelled as 'bug,' within the last 12 hours from now.

UUID **id**: A unique identifier for the metric. This should be automatically generated and incremented.

Integer **user_id**: id of user that creates matrix.

String **name**: The name of the metric. For example, in this case, it could be "bug_10_12".

Enum **label**: The label type associated with the tasks you want to track. There are 9 possible types.

Integer **threshold**: The quantity or count of the specific label required to meet the condition.

Integer **time_frame_hours**: The time range in hours over which the tasks should be checked (1-24).

Label : (enum)

- 1- **bug**: Something isn't working
 - 2- **documentation**: Improvements or additions to documentation
 - 3- **duplicate**: This issue or pull request already exists
 - 4- **enhancement**: New feature or request
 - 5- **good_first_issue**: Good for newcomers
 - 6- **help_wanted**: Extra attention is needed
 - 7- **invalid**: This doesn't seem right
 - 8- **question**: Further information is requested
 - 9- **wontfix**: This will not be worked on
-

Action Services:

id	user_id	name	create_date	condition	to	action_type	run_on_time	run_on_day	message	is_enabled	is_deleted	last_update	last_run
1	1001	Help developer	12.04.2026	[[1]]	0509999999	sms	14:00	Sunday	"Help me..."	true	false	-	2026_05_22T11_30_00
2	1001	fix bugs	12.04.2026	[[1,2],[3]]	ameen@gmail.com	email	15:30	All	"Need to fix"	false	true	2024_05_22T11_30_00	2026_05_22T11_30_00

UUID **id**: A unique identifier for the action. It should be automatically generated and incremented.

String **owner_id**: The identifier for the manager of the project associated with the action.

String **name**: The name of the action.

Date **create_date**: The date when the action was created.

Enum **action_type**: The type of action. There are two possible types: **SMS** and **EMAIL**.

Time **run_on_time**: The time when the action is scheduled to run. The time must be selected from full hours or half hours only (e.g., 14:00, 14:30, 15:00, etc.). The value should be within the range of **00:00** to **24:00**.

Time **run_on_day**: The day of the week when the action is scheduled to run (e.g., Sunday, Monday, etc.). There should also be an option to select "All" days if the action needs to run every day of the week.

String **message**: The content of the message that you want to send when the action is triggered

String **to**: info to send when the condition returns true.

Boolean **is_enabled**: A flag indicating whether the action is active and needs to be checked

Boolean **is_deleted**: A flag indicating whether the action has been deleted from the system. Even if deleted, the action should be saved in the database.

Timestamp **last_update**: The timestamp of the last update made to the action. This should be updated any time there is a change in the action, such as toggling the **is_enabled** flag.

Timestamp **last_run**: The timestamp of the last time the action was checked. This should be updated every time the action is checked.

Array of Array of Integers **[[int,int],[int]] condition**:

Description:

- This field represents conditions in a matrix format. Each condition is built from individual metrics (defined in the metrics table) combined using logical operators like **AND** and **OR**.

Structure:

- The outer array represents groups of conditions.
- Each inner array contains the IDs of metrics.
- The metrics within an inner array are combined using the **AND** operator.
- Different inner arrays are combined using the **OR** operator.

Building Blocks:

- Metrics**: These are predefined in the metrics table and are identified by their unique IDs.
- Operators**:
 - AND**: All metrics within the same inner array must satisfy the condition.
 - OR**: At least one of the inner arrays (groups of conditions) must satisfy the condition.

Example:

- [[1, 2], [3]]**

- **Interpretation:**
 - **Group 1 ([1, 2]):** Both Metric 1 **AND** Metric 2 must be satisfied.
 - **Group 2 ([3]):** Metric 2 alone must be satisfied.
 - **Overall Condition:**
 - The condition is satisfied if either:
 - Both Metric 1 **AND** Metric 2 are true (Group 1).
 - Or, Metric 3 alone is true (Group 2).
-

Job Scheduler:

The job scheduler runs every 30 minutes, checking for actions that need to be performed at the current hour and day of the week. It ensures that the actions meet the following conditions:

- **is_enabled Flag:** The action must be enabled (`is_enabled = true`).
- **is_deleted Flag:** The action must not be marked as deleted (`is_deleted = false`).

If all these conditions are satisfied, the scheduler places the action into the queue for execution.

Processor Services:

The **Processor** microservice performs the following tasks:

Action Retrieval:

It pulls actions from the queue.

Condition Query:

Queries the **platformInformation** database using the conditions specified for the metric in each action to check if the threshold is met.

Threshold Verification:

If the threshold is met:

Kafka Message Production:

Produces a Kafka message to one of two topics based on the action's type:

- 1- **email Topic:** For actions that require email notifications.
- 2- **sms Topic:** For actions that require SMS notifications.

The Kafka message includes the recipient's email or phone number and the message content retrieved from the action.

Notification Queue Insertion:

Inserts the message into the notification queue for further processing.

This microservice ensures that actions are only processed if they meet the required conditions and thresholds, and subsequently sends notifications to the appropriate users.

Notification Services:

The Notification Services consist of two components: SMS and Email. Each service operates as follows:

1. **Queue Pulling:**
 - Each notification service pulls messages from the queue based on the action type (SMS or Email).
2. **Notification Sending:**
 - **SMS Service:**
 - Pulls messages from the queue designated for the SMS topic.
 - Sends SMS notifications using the defined text from the action.
 - **Email Service:**
 - Pulls messages from the queue designated for the Email topic.
 - Sends email notifications using the defined text from the action.

These services handle the delivery of notifications according to the action type and ensure that messages are sent to the appropriate recipients.

Evaluation Services:

The Evaluation Services are designed to help managers gain insights into developers' performance and workload by querying the `platformInformation` database.

Endpoints:

1. **Find the Developer with the Most Occurrences of a Specific Label Within a Specific Time Frame.**
`GET /evaluation/developer/most-label?label={labelName}&since={day}`
 - **Description:**
 - Retrieves the developer with the highest number of tasks associated with a specific label.
 - **Query Parameter:**
 - `label={labelName}` - The label you want to search for (e.g., "bug," "feature").
 - `since={day}` - The number of days before the current date for which you want to check the task count.
2. **Aggregation of Each Label for the Specified Developer Within a Specific Time Frame.**
`GET /evaluation/developer/{developer_id}/label-aggregate?since={day}`

- **Description:**
 - Returns the count of tasks associated with each label for the specified developer.
 - **Path Parameter:**
 - `{developer_id}` - The ID of the developer whose label counts you want to retrieve.
 - **Query Parameter:**
 - `since={day}` - The number of days before the current date for which you want to check the task count.
3. **Get the Total Number of Tasks Assigned to a Specified Developer Within a Specific Time Frame.**
- `GET /evaluation/developer/{developer_id}/task-amount?since={day}`
- **Description:**
 - Returns the total number of tasks assigned to a specified developer within the last specified number of days.
 - **Path Parameter:**
 - `{developer_id}` - The ID of the developer.
 - **Query Parameter:**
 - `since={day}` - The number of days before the current date for which you want to check the task count.

Once the result is obtained from the endpoint, it should be sent to the notifications queue with the "email" topic. This ensures that the manager receives the information via email.

Logger Services:

Database Schema (MongoDB):

Each log entry is stored as a document in a MongoDB collection named `logs`. The schema may include the following fields:

- `_id`: Unique identifier for each log entry.
- `timestamp`: The date and time when the log entry was created.
- `serviceName`: The name of the microservice that generated the log.
- `logLevel`: The severity level of the log (e.g., INFO, DEBUG, WARN, ERROR).
- `message`: The log message.

Logging Requirements:

- **Processor Microservice**: Logs should be saved when inserting messages into Kafka.
- **Email and SMS Microservices**: Logs should be saved when sending messages, indicating whether the operation was successful or failed.

Implementation with SLF4J

In your microservices, you'll use SLF4J for logging and send the logs to a dedicated logging microservice that saves them in MongoDB.

Part Two: Deployment

When the development is complete, we need to deploy the microservice to Minikube:

1. **Build the Docker image:**
 - Create a Dockerfile for the microservice.
 - Build the Docker image locally using the command `docker build -t <image-name> ..`
2. **Push the Docker image to Docker Hub:**
 - Log in to Docker Hub using the command `docker login`.
 - Tag the image with your Docker Hub username and repository: `docker tag <image-name> <your-dockerhub-username>/<repository-name>:<tag>`.
 - Push the image to Docker Hub using `docker push <your-dockerhub-username>/<repository-name>:<tag>`.
3. **Start Minikube:**
 - Launch Minikube using the command `minikube start`.
4. **Deploy to Minikube:**
 - Use `kubectl` to deploy the microservice to Minikube:
 - Create a Kubernetes deployment file (YAML) that includes specifications for your microservice (e.g., container image, replicas).
 - Deploy the microservice using `kubectl apply -f <deployment-file.yaml>`.
5. **Expose the service:**
 - Expose the deployment to make the microservice accessible:
 - Use a `Service` in Kubernetes to expose the microservice.
 - If necessary, create an `Ingress` to manage external access.
6. **Verify the deployment:**
 - Check the status of the pods and services using `kubectl get pods` and `kubectl get services`.
 - Ensure that the microservice is running correctly in Minikube.
7. **Access the microservice:**
 - Use `minikube service <service-name>` to get the URL for accessing the microservice.
8. **Test the system using the frontend or Postman:**

- Use the frontend application or Postman to send requests to the microservice.
- Verify that the microservice behaves as expected, responding correctly to API calls and interacting with the other components in the system.